

Diagrama de clase - Sistem CRUD cu API Gateway si microservicii

The diagram illustrates the architecture of a CRUD system using an API Gateway and four microservices. The components and their interactions are as follows:

- API Gateway - port 8080**
 - «SpringBootApplication» ApiGatewayApp**: Contains `+main(args : Array~String~) : Unit`. It performs a `component scan` on the `GatewayService`.
 - «Service» GatewayService**: Contains `-createUrl : String`, `-readUrl : String`, `-updateUrl : String`, `-deleteUrl : String`, `-restTemplate : RestTemplate`, and `-jsonHeaders() : HttpHeaders`. It `trimite prin HTTP` to the `IGatewayService`.
 - «RestController» GatewayController**: Contains `+initTable() : ResponseEntity~Any~`, `+addBeer(beer : Beer) : ResponseEntity~Any~`, `+getAllBeers() : ResponseEntity~Any~`, `+getBeerById(id : Int) : ResponseEntity~Any~`, `+getBeersByName(name : String) : ResponseEntity~Any~`, `+getBeersByPrice(maxPrice : Float) : ResponseEntity~Any~`, `+updateBeer(id : Int, beer : Beer) : ResponseEntity~Any~`, and `+deleteBeer(id : Int) : ResponseEntity~Any~`. It `injecteaza` the `IGatewayService` and `primeste / returneaza` data from the `Beer` data class.
 - IGatewayService**: Contains `+initTable() : Any`, `+addBeer(beer : Beer) : Any`, `+getAllBeers() : Any`, `+getBeerById(id : Int) : Any`, `+getBeersByName(name : String) : Any`, `+getBeersByMaxPrice(maxPrice : Float) : Any`, `+updateBeer(id : Int, beer : Beer) : Any`, and `+deleteBeer(id : Int) : Any`. It `injecteaza` the `Beer` data class.
 - «data class» Beer**: Contains `+id : Int`, `+name : String`, and `+price : Float`.
- Create Microservice - port 8081**
 - «SpringBootApplication» CreateServiceApp**: Contains `+main(args : Array~String~) : Unit`. It performs a `component scan` on the `CreateService`.
 - «Service» CreateService**: Contains `-repository : IBeerRepository` and `-safePattern : Pattern`. It `injecteaza` the `IBeerRepository` and `valideaza si salveaza` data.
 - «RestController» CreateController**: Contains `+createService : ICreateService` and `+addBeer(beer : Beer) : ResponseEntity~Any~`. It `injecteaza` the `CreateService` and `primeste / returneaza` data.
 - IBeerRepository**: Contains `+initTable() : Unit`, `+addBeer(beer : Beer) : Unit`, `+getById(id : Int) : Beer?`, `+getByName(name : String) : Beer?`, `+update(beer : Beer) : Unit`, and `+delete(id : Int) : Unit`. It `mappeaza datele` to the `Beer` data class.
 - «data class» Beer**: Contains `+id : Int`, `+name : String`, and `+price : Float`.
 - ICreateService**: Contains `+initTable() : Unit` and `+addBeer(beer : Beer) : Beer`. It `injecteaza` the `Beer` data class.
- Read Microservice - port 8082**
 - «SpringBootApplication» ReadServiceApp**: Contains `+main(args : Array~String~) : Unit`. It performs a `component scan` on the `ReadService`.
 - «Service» ReadService**: Contains `-repository : IBeerRepository`. It `injecteaza` the `IBeerRepository` and `cauta si filtreaza` data.
 - «RestController» ReadController**: Contains `+readService : IReadService` and `+getById(id : Int) : ResponseEntity~Any~`. It `injecteaza` the `ReadService` and `returneaza` data.
 - IReadService**: Contains `+getAll() : List~Beer~`, `+getById(id : Int) : Beer?`, `+getByName(name : String) : Beer?`, and `+getByMaxPrice(maxPrice : Float) : List~Beer~`. It `injecteaza` the `Beer` data class.
 - IBeerRepository**: Contains `+initTable() : Unit`, `+addBeer(beer : Beer) : Unit`, `+getById(id : Int) : Beer?`, `+getByName(name : String) : Beer?`, `+update(beer : Beer) : Unit`, and `+delete(id : Int) : Unit`. It `mappeaza datele` to the `Beer` data class.
 - «data class» Beer**: Contains `+id : Int`, `+name : String`, and `+price : Float`.
- Update Microservice - port 8083**
 - «SpringBootApplication» UpdateServiceApp**: Contains `+main(args : Array~String~) : Unit`. It performs a `component scan` on the `UpdateService`.
 - «Service» UpdateService**: Contains `-repository : IBeerRepository` and `-safePattern : Pattern`. It `injecteaza` the `IBeerRepository` and `valideaza si actualizeaza` data.
 - «RestController» UpdateController**: Contains `+updateService : IUpdateService` and `+updateBeer(id : Int, beer : Beer) : ResponseEntity~Any~`. It `injecteaza` the `UpdateService` and `primeste` data.
 - IUpdateService**: Contains `+updateBeer(id : Int, beer : Beer) : Boolean`. It `injecteaza` the `Beer` data class.
 - IBeerRepository**: Contains `+initTable() : Unit`, `+addBeer(beer : Beer) : Unit`, `+getById(id : Int) : Beer?`, `+getByName(name : String) : Beer?`, `+update(beer : Beer) : Unit`, and `+delete(id : Int) : Unit`. It `mappeaza datele` to the `Beer` data class.
 - «data class» Beer**: Contains `+id : Int`, `+name : String`, and `+price : Float`.
- Delete Microservice - port 8084**
 - «SpringBootApplication» DeleteServiceApp**: Contains `+main(args : Array~String~) : Unit`. It performs a `component scan` on the `DeleteService`.
 - «Service» DeleteService**: Contains `-repository : IBeerRepository`. It `injecteaza` the `IBeerRepository` and `verifica existenta` data.
 - «RestController» DeleteController**: Contains `+deleteService : IDeleteService` and `+deleteBeer(id : Int) : ResponseEntity~Any~`. It `injecteaza` the `DeleteService` and `foloseste id-ul` to the `IDeleteService`.
 - IDeleteService**: Contains `+deleteBeer(id : Int) : Boolean`. It `injecteaza` the `Beer` data class.
 - IBeerRepository**: Contains `+initTable() : Unit`, `+addBeer(beer : Beer) : Unit`, `+getById(id : Int) : Beer?`, `+getByName(name : String) : Beer?`, `+update(beer : Beer) : Unit`, and `+delete(id : Int) : Unit`. It `mappeaza datele` to the `Beer` data class.
 - «data class» Beer**: Contains `+id : Int`, `+name : String`, and `+price : Float`.
- Database**
 - «database» SQLite Database**: Contains `+path : String = "/tmp/beer.db"` and `+table : String = "beers"`. It is connected to the `Beer` data class via `JDBC`.

Interactions:

- The `API Gateway` receives HTTP requests (CREATE, READ, UPDATE, DELETE) and routes them to the corresponding microservices.
- Each microservice interacts with the `Beer` data class and the `SQLite Database` to perform CRUD operations.
- The `API Gateway` also performs a `component scan` on the `GatewayService` to register the microservices.

Annotations:

- Ruteaza cererile HTTP:** CREATE -> port 8081, READ -> port 8082, UPDATE -> port 8083, DELETE -> port 8084.
- API Gateway este singurul punct de intrare pentru client.**
- Endpoint de baza: /api/beer**
- Valideaza numele berii si insereaza o inregistrare noua in baza de date.**
- Citeste toate berile sau cauta dupa:** - id, - nume, - pret maxim.
- Verifica existenta berii, valideaza numele si actualizeaza inregistrarea existenta.**
- Verifica existenta berii, apoi sterge inregistrarea dupa identificator.**
- Baza SQLite este comuna celor patru microservicii CRUD.**
- Tabela: beers(id, name, price)**

